
THESIS TITLE

A PREPRINT

Knud Rasmus Sønderbek

April 13, 2026

ABSTRACT

Some super cool abstract of all my findings

1 Introduction

Neural networks continue to achieve remarkable performance across domains ranging from medical diagnosis to autonomous systems. This increase in performance has, however, come with a corresponding increase in complexity, making these systems less interpretable to those who build and deploy them. As neural networks enter high-stakes environments, where errors carry legal, financial, or human cost, interpretability becomes a necessity. The concern is not merely technical but ethical: a fundamental principle of accountability holds that the creator of a tool can only be held responsible for its outcomes to the extent that those outcomes could have been reasonably foreseen. A system whose internal reasoning cannot be understood is, by definition, one whose failures cannot be anticipated.

Mechanistic interpretability aims to reverse-engineer the internal computations of neural networks into their simplest forms, such that they can be expressed as humanly interpretable algorithms.

Many of the current popular methods within this space, such as sparse autoencoders [Cun+23], circuit analysis, and probing, primarily operate in activation space. However, each of these methods comes with its own set of limitations. Sparse autoencoders treat features as independent directions and discard the geometric structure between them, despite growing evidence that this structure carries meaningful information (Leask et al., 2025; Mendel, 2024; Engels et al., 2024). Moreover, sparse autoencoders are trained to reconstruct activations and do not to explicitly identify the computational units that produced them. [Lea+25; Cha+25] The resulting decomposition may be functionally equivalent at a given layer, while not resembling the mechanisms that the network employs. Circuit-level methods face the deeper issue that neural networks are densely connected, meaning any given output can be equally well attributed to a multitude of causal pathways, leaving the choice of circuit underdetermined.

A more recent line of work shifts the object of decomposition from activations to parameters. More concretely, these methods aim to decompose a network’s weights into a sum of simpler components, each of which corresponds to a concrete “mechanism”. This reframing sidesteps several of the issues above: mechanisms are no longer constrained to specific architectural components, such as neurons or attention heads, and can instead be distributed across multiple architectural components. Moreover, they decompose the network into functional units rather than representational directions, which may further enhance their interpretability.

Recently proposed methods guided by the principle of minimum description length have shown promising results in this direction, aiming to decompose a network into parameter components that are faithful to the original weights, minimal in the number required per input, and individually as simple as possible.

One such method, Stochastic Parameter Decomposition (Bushnaq et al., 2025), decomposes networks into rank-one subcomponents. Since not all components are relevant for every input, the method learns a function that estimates each component’s causal importance for a given input. However, each component’s importance is calculated independently, relying only on its own activation. As noted in the original work, this is likely too restrictive for non-toy models, as it prevents each component’s importance from being informed by the state of other components. This work replaces the independent importance functions with a graph neural network that determines each component’s importance conditioned on its relationships to all other components, while preserving explainability through its generalised additive structure. Additionally the use of an additive importance function, generalizes the decomposition to k-dimensional components. This generalization is motivated by the growing evidence, that transformer models, learn features which occupy irreducible multidimensional subspaces.

[Bra+25]

2 Related work

This is the related work of the thesis.

3 Method

3.1 Background on APD

The initial inspiration for shifting the focus from activations to parameters came from Attribution-based Parameter Decomposition (APD) [Bra+25]. APD decomposes the weights of a network into a set of parameter components: vectors in parameter space that satisfy three desirable properties.

Let $f(x, W)$ be a neural network with weight matrices $W = \{W^1, \dots, W^L\}$. A set of parameter components should satisfy the following properties:

- **Faithfulness:** The parameter components should sum to the parameters of the original network.
- **Minimality:** As few parameter components as possible should be required to replicate the network’s output on any given input.
- **Simplicity:** Each component should span as few weight matrices and as few ranks as possible.

If a set of parameter components satisfies these properties, they are said to comprise the network’s mechanisms [Bra+25].

However, APD suffers from several practical limitations. Each parameter component is a full vector in parameter space $\mathbb{R}^{|\theta|}$, where $|\theta|$ is the total number of parameters in the network, thus increasing the memory cost to $O(\theta \cdot C)$, where C is the number of components, making the method computationally expensive to scale to larger models. APD is also highly sensitive to its top- k hyperparameter, which specifies the expected number of active components per input and must be chosen in advance. Lastly, APD relies on the magnitude of the gradient as a proxy for the causal importance of each component. These gradients have been shown to be poor approximations of the true importance. (cite the papers here).

3.2 Stochastic parameter decomposition

Stochastic Parameter Decomposition (SPD) was introduced as a solution to the limitations of APD. SPD decomposes each weight matrix into a sum of rank-one matrices, called subcomponents:

$$W^l \approx \sum_{c=1}^C \vec{u}_c^l \vec{v}_c^{lT}, \quad \text{where } \vec{u} \in \mathbb{R}^{d_{out}}, \vec{v} \in \mathbb{R}^{d_{in}}$$

The number of subcomponents C is a hyperparameter that may be chosen to be larger than the rank of W^l enabling the method to learn features in superposition [BBS25].

These subcomponents are then trained to be faithful to the original weights, to have as few active subcomponents as possible for any given input, and lastly to reconstruct the original network’s output.

3.2.1 Faithfulness

3.2.2 Stochastic reconstruction

3.2.3 Minimality

3.2.4 Limitations

These components are later clustered manually.... -¿ GNaN solves Components are rank 1, many features are multidimensional, dependent components are trained independently

3.3 Some clever name for my additions

This works primary purpose is to address two methodological limitations of SPD, namely the rank-one constraint on subcomponents and the independence of the importance functions across subcomponents. The necessity of these modifications will be demonstrated in the experiments, where dependent and multi-dimensional features will be shown to be common in even simple models, and the original SPD method will be shown to fail to recover them (i hope).

Additionally, this work also provides a first step towards a general clustering mechanism that operates during training rather than a post-hoc step, by masking features of each subcomponent independently and then allowing related features to be clustered together (again i hope - or the circuit discovery thing..? However it needs modifications).

To achieve this, three modifications are introduced: rank- k subcomponents that can natively capture multi-dimensional mechanisms, per-dimension ablation that implicitly learns the rank of each mechanism, and an additive graph neural network based importance function that conditions each importance on the activation of neighbouring subcomponents.

3.3.1 Rank- k subcomponents

In SPD, each subcomponent is a rank-one outer product of two vectors. A mechanism of rank r must therefore be represented as r separate subcomponents, with no training signal to keep them coherent. To generalize this, each pair of vectors is replaced with a pair of matrices:

$$W^l \approx \sum_{c=1}^C U_c^l V_c^{l\top}, \quad \text{where } U_c^l \in \mathbb{R}^{d_{\text{out}} \times k}, V_c^l \in \mathbb{R}^{d_{\text{in}} \times k}$$

Each subcomponent $U_c^l V_c^{l\top} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a matrix of rank at most k . Setting $k = 1$ yields the original SPD definition. As a result of this change, the inner activation used in the importance function becomes a k -dimensional vector:

$$\mathbf{h}_c^l(x) = V_c^{l\top} \vec{a}^l(x) \in \mathbb{R}^k$$

The extension from SPD follows naturally: the importance function is no longer derived from a single inner activation, but from k inner activations, which are processed jointly to produce an overall importance value for the subcomponent.

3.3.2 GNAN-based importance functions

In SPD, each importance function γ_c^l is an independent MLP that receives only its own inner activation. Intuitively, however, causal importance is relational: whether a subcomponent can be ablated without affecting the output may well depend on which other subcomponents are active alongside it.

To address this, the independent MLPs are replaced with a Graph Neural Additive Network (GNAN) that can relate each subcomponent to its neighbourhood.

The C subcomponents are treated as nodes in a graph, with node features given by their inner activation vectors $\mathbf{h}_c^l(x) \in \mathbb{R}^k$. The GNAN updates each node's representation through an additive neighbourhood aggregation. For node i in layer l and feature dimension k ¹:

$$[\mathbf{z}_i]_k = \sum_{j=1}^N \frac{1}{|\mathcal{L}(j)|} \cdot \rho\left(\frac{1}{1 + \text{dist}(j, i)}, \cos(j, i)\right) \cdot f_k([\mathbf{h}_j]_k)$$

OBS: Implementation ill present today is this:

$$\mathbf{z}_i = \sum_{j=1}^N \rho(\text{layer distance, cosine similarity}) \cdot f_k([\mathbf{h}_j]_k)$$

where f_k is a learned function applied to the k -th inner activation of each neighbouring node j ; $|\mathcal{L}(j)|$ is the number of nodes in the same layer as j , which normalises the contribution of each layer; $\text{dist}(j, i)$ is the layer distance between nodes j and i ²; $\cos(j, i)$ is the cosine similarity between their parameter vectors; and ρ is a learned function that maps both the distance kernel and the cosine similarity to a scalar weight.

$\mathbf{z}_i$ becomes a vector in $\mathbb{R}^k$, which fits perfectly with the current activation vectors.

¹I am very open to alternative equations, i.e perhaps we delete the layer distance inside rho, however it seemed necessary to use cosine similarity, as otherwise the signal was useless as described in some earlier section

²The layer distance, could later be signed to differentiate between previous and future layers

A separate MLP is trained to combine the aggregated neighbourhood information and the node's own activation into an importance value in $[0, 1]$: Producing a single importance value per subcomponent and passing it through a leaky hard sigmoid, following SPD [BBS25]:

$$g_c^l = \sigma_H(AGG(\mathbf{z}_i^l, \mathbf{h}_i^l))$$

The self-connection through f_{self} (or just the raw activation) ensures that each node retains its own identity after aggregation, mitigating the oversmoothing that can occur in GNNs. {Clever cite here, i.e gnn book}

Alternative formulation, that would work with the next section

After the GNAN update, the per-dimension importance values are computed from the updated representation and passed through a leaky hard sigmoid, following SPD [BBS25]:

$$\mathbf{g}_c^l(x) = \sigma_H(\mathbf{h}_c^l(x)) \in [0, 1]^k$$

in other words, the importance function becomes multidimensional, and each column can be masked independently.

Remarks:

For now the edge set construction is left as the fully connected graph, which remains reasonably computationally expensive for small models. However it could very easily be made more efficient, for other architectures, or by considering only the k-hop neighbourhood of each node, or only considering prior neighbours or later neighbourhoods. Likewise we could store both distances as sparse structures as they are symmetric. $\cos(j, i) = \cos(i, j)$

Remark 2:

The choice of edge set construction and distance function is a very important design choice, e.g using the layer distance on a fully connected graph encodes the same signal for all nodes in a layer: For nodes i_1, i_2 in the same layer l :

$$\forall j: \quad \text{dist}(j, i_1) = \text{dist}(j, i_2) = |k_j - l|$$

and therefore

$$[\mathbf{h}_{i_1}]_k = \sum_{j=1}^N \frac{1}{\#\text{dist}_{i_1}(j, i_1)} \cdot \rho\left(\frac{1}{1 + |k_j - l|}\right) \cdot f_k([\mathbf{x}_j]_k) = [\mathbf{h}_{i_2}]_k$$

since no term in the sum depends on the identity of i within layer l , only on the layer index l itself.

Additionally this produces a counterproductive signal, the gnan tries to boost or suppress all nodes in a layer together, instead of learning to differentiate between them.

Remark 3:

The choice of distance should probably be reconsidered in general, I have a feeling that the function needs to be from an asymmetric metric space, i.e

$$\text{dist}(i, j) \neq \text{dist}(j, i)$$

Atleast if the intuition is if component A in layer k, is highly active, perhaps component B in layer k+1 should likely be inactive, as the other component needs to handle this feature, thus the distance from A to B should be different, as otherwise both are boosted or suppressed together.

Remark 4:

The current experiments are more sensitive to hyperparameters, than the original code seems to be, if the minimality pressure is too low, then all components are active, and doesn't "kill" any of them

Remark 5:

I haven't been able to find any "benefit" in setting k higher, this is ofcourse because the ground truth of all the models we have are rank 1 independent features, we need to find a scenario where the dependence and or rank k is actually necessary:

My ideas

- Sample points from the unit circle, i.e $\cos(\theta)^2 + \sin(\theta)^2 = 1$ then

$$x = \sqrt{1 - \sin(\theta)^2} \text{ or } -\sqrt{1 - \sin(\theta)^2}$$

i.e something can only be expressed with a rank 2 component?

- we could look for circuits, the goal is then no longer to find N independent features, but suppose we have input groups $[0,1,2]$, $[2,3,4]$, $[3,4,5]$ then we want one component that activates for the first group, one for the second and one for the third

Remark 6:

I added the gnan plots to wandB remember to show them

Remark 7:

The per feature construction with only 1 feature is rather boring, and does not utilize the expressiveness at all.

Remark 8:

I dont think its a code problem, as i can reproduce equivalent results to their findings. Im just not able to beat them. We need to discuss experiments where the k dimensionality and dependence actually matters.

3.3.3 Per-dimension ablation and implicit rank selection (Hypothesis: ask lukas)

Following the extension to rank- k subcomponents, and the alternative formulation of the GNAN, the output $g_c^l \in [0, 1]^k$ is a per feature importance value, and thus the masking can be applied independently to each feature. The intuition behind this is that if a mechanism captures a 1 dimensional feature, it would only need of the k dimensions to be active.

Likewise if this actually ends up working, k can be considered an upper bound or a so called "capacity" of a mechanism, it also follows nicely with the descriptive length principle found in the appendix of APD.

To determine the effective rank of each subcomponent, the stochastic masking becomes:

$$[m_c^l(x, r)]_j = [g_c^l(x)]_j + (1 - [g_c^l(x)]_j)r_{c,j}^l, \quad r_{c,j}^l \sim U(0, 1)$$

for each j in $1, \dots, k$

Notably this treats each feature independently, which may not be desirable, as such we could modify it ever so slightly:

$$[m_c^l(x, r)]_j = [g_c^l(x)][g_c^l(x)]_j + (1 - [g_c^l(x)][g_c^l(x)]_j)r_{c,j}^l, \quad r_{c,j}^l \sim U(0, 1)$$

The signal becomes an importance of two factors, if the component is important and both features are important, it remains active, if only one feature is important, the other will be pushed to zero

The masked weight matrix becomes:

$$W^l(x, r) = \sum_{c=1}^C U_c^l \text{diag}(\mathbf{m}_c^l(x, r)) V_c^{l\top}$$

Where the diagonal matrix is the per-feature mask for subcomponent c .

The importance minimality loss penalises all k dimensions of all subcomponents toward zero:

$$\mathcal{L}_{\text{import}} = \sum_{l=1}^L \sum_{c=1}^C \sum_{j=1}^k |[g_c^l(x)]_j|^p$$

This follows the same logic by which SPD selects the number of active subcomponents: the minimality loss drives unused components toward zero, and only those that are genuinely required to reconstruct the target model's output survive.

This makes k a soft upper bound rather than a sensitive hyperparameter. In practice, k can be set conservatively large and the effective rank of each subcomponent is learned from the data.

4 Benefits

4.1 Dead salmon thing

5 Experiments

5.1 Toy model of superposition

5.2 Rank- r Toy Model of Superposition

5.3 Toy model with relu

5.4 Smallest possible LM

6 Discussion

7 References

8 Other things to consider?

Minimal descriptive length?

8.1 That some features are multidimensional

Think back to SAE, that learn a dictionary of features, then find the linear combination of those features, however that this fails for cyclical features, as its cheaper to learn a single feature, than learn a direction and a periodic function. Now for your experiment, the dream scenario is that we allow a multi dimensional component, and that this can learn the cyclical feature, i.e a weekly cycle or modular arithmetic either do this or extend TMS to be multi-dimensional, show if it works better with or gnn or not, likewise we need to show that through individual masking of features, we can teach a component to kill one of its features, if it is better explained by a rank 1 component than rank 2.

- [BBS25] Lucius Bushnaq, Dan Braun, and Lee Sharkey. *Stochastic Parameter Decomposition*. 2025. arXiv: 2506.20790 [cs.LG]. URL: <https://arxiv.org/abs/2506.20790>.
- [Bra+25] Dan Braun et al. *Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition*. 2025. arXiv: 2501.14926 [cs.LG]. URL: <https://arxiv.org/abs/2501.14926>.
- [Cha+25] David Chanin et al. *A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders*. 2025. arXiv: 2409.14507 [cs.CL]. URL: <https://arxiv.org/abs/2409.14507>.
- [Cun+23] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: <https://arxiv.org/abs/2309.08600>.
- [Lea+25] Patrick Leask et al. *Sparse Autoencoders Do Not Find Canonical Units of Analysis*. 2025. arXiv: 2502.04878 [cs.LG]. URL: <https://arxiv.org/abs/2502.04878>.